

kharcha

ML Learning & Build Plan

for Chetan · April 2026

Goal	Fine-tune a small transformer to parse Indian bank SMS → structured JSON. Ship it inside the kharcha React Native app.
Your baseline	Strong frontend (React / TS / JS). Used Silero VAD via onnxruntime in a browser context. Zero ML training or model-export experience.
Time budget	~2–2.5 hrs/day on weekdays (5–6 PM + 10–11:30 PM), ~3 hrs/day weekends. Roughly 14–16 hrs/week.
Total duration	8 weeks to a shipped, working on-device parser.
Runtime decision	onnxruntime-react-native (not ExecuTorch) — matches your existing mental model and is more mature.

This plan is written for a starting-from-zero ML learner who ships production code for a living. Theory is included only where it directly prevents you from debugging your own pipeline. Every week maps to real study slots in your existing schedule.

1. architecture decisions

Fix these three things before writing a single line of ML code. Getting them wrong wastes weeks.

1.1 runtime: onnxruntime-react-native, not ExecuTorch

Factor	ExecuTorch	ONNX Runtime
Package	react-native-executorch	onnxruntime-react-native
Model format	.pte (PyTorch-native)	.onnx
Export tool	torch.export() + ExecuTorch	optimum-cli (1 command)
Maturity	Newer, docs sparse	Mature, widely documented
Your prior XP	None	Used in HackerEarth proctoring
Custom models	Less documented	Extensive guides

The green column is your path. Your ONNX experience from HackerEarth is contextual familiarity only — you loaded a pre-built model, you did not export or fine-tune one. That is fine; ONNX export is one CLI command you will learn in Week 6.

1.2 model: TinyBERT or MiniLM, not DistilBERT or T5

Model	Size (quantised INT8)	Verdict
T5-small	~80 MB (3 ONNX files)	Over budget + slow autoregressive decoding on mobile
DistilBERT	~63–66 MB	Just over 50 MB limit; still too large
TinyBERT	~15 MB	Best choice — fast, fits easily
MiniLM-L6	~22 MB	Good fallback; slightly more accurate than TinyBERT

1.3 parse strategy: regex-first, ML as fallback

Start with [transaction-sms-parser](#) (npm package, JS/TS). It handles Axis, ICICI, HDFC, SBI, Kotak, and 15+ other Indian banks with regex, outputting {account, transaction, balance} JSON. It is already done, already tested, already covers most of your users. Add your ML model as a fallback for SMS formats it cannot recognise. This is the correct engineering call — ML adds value at the margin, not as a replacement for deterministic parsing.

2. skill map

Five ML skills stand between you and a shipped kharcha parser. Everything else is either something you already know or something you pick up from documentation.

Python for ML

Time: 1–2 days

Slot: Weekends,
Week 1

You know JS/TS deeply. Python syntax is learnable in a weekend. You need just enough to run training scripts, manipulate DataFrames, and call library APIs — not to write production Python.

Transformer architecture (conceptual)

Time: 4–6 hrs

Slot: Week 1–2

You need to understand encoder models (BERT family) vs decoder models (GPT family). Bank SMS parsing is a classification/NER task, so you need an encoder. You do not need to understand every attention head — just enough to know why you are choosing TinyBERT over T5.

Fine-tuning for NER (token classification)

Time: 20–25 hrs

Slot: Weeks 2–4

The core skill. Loading a pretrained model, attaching a classification head, feeding labelled data, running a training loop. This is the bulk of what the HuggingFace course teaches. For bank SMS, each token gets a BIO label: B-AMOUNT, I-AMOUNT, B-ACCOUNT, B-TYPE, O (other).

ONNX export and quantisation

Time: 4–6 hrs

Slot: Week 6

Converting your fine-tuned PyTorch model to ONNX format, then shrinking it to INT8 with quantisation. Handled almost entirely by two tools: HuggingFace Optimum (export) and onnxruntime.quantization (shrink). One CLI command + three lines of Python.

onnxruntime-react-native integration

Time: 6–8 hrs

Slot: Week 7–8

Loading the ONNX model in Expo/React Native, running inference, and post-processing the NER output (collecting BIO-tagged tokens into structured JSON fields). The inference call is ~10 lines. Post-processing is TypeScript you already know how to write.

3. 8-week plan

Based on your schedule: 2–2.5 hrs/day on weekdays (5–6 PM or 10 PM–midnight), ~3 hrs/day on weekends. Total ~14–16 hrs/week. Training days (boxing/gym) reduce the evening slot — plan deeper study for 10 PM blocks or weekend mornings.

week 0 (pre-work, ~4 hrs) <i>before you start the course material</i>		
Install transaction-sms-parser	~2 hrs	weekend
Run <code>npm install transaction-sms-parser</code> in the <code>kharcha</code> frontend. Wire it up so it parses incoming SMS. Test on 10–15 real messages from your bank. This is your production parser — ML fills the gaps it cannot handle.		
Collect raw SMS data	~1 hr	anytime
Export 50–100 bank SMS messages from your phone. Axis, HDFC, ICICI — whatever banks you actually use. Save them as a text file. You will label these in Weeks 3–4. Starting early means you are not blocked later.		
Set up Python environment	~1 hr	weekend
Install Python 3.11 via <code>pyenv</code> or Homebrew on your Mac Mini. Run: <code>pip install torch transformers datasets accelerate</code> . Open a Jupyter notebook in VS Code. Confirm you can run <code>import transformers</code> without errors. That is the entire setup.		
week 1 (foundation, ~14 hrs) <i>python basics + transformer mental model</i>		
Python crash course for JS devs	~4 hrs	weekend mornings
Do not do a full Python course. Read this one resource only: 'Python for JavaScript Developers' cheatsheet (search this exact phrase, multiple good results exist). Focus on: list comprehensions, dictionaries, f-strings, <code>enumerate()</code> , <code>zip()</code> , and basic file I/O. You do not need classes yet.		
fast.ai Practical Deep Learning — Lesson 4 only	~8 hrs	split across week
Go to course.fast.ai , watch Lesson 4 (NLP). This is top-down: you see a working NLP model before touching any theory. Jeremy Howard's teaching style works well for engineers — he shows you the whole pipeline first. Do not do Lessons 1–3 unless you want to.		
HF Course Chapter 1 — <code>pipeline()</code> API	~2 hrs	2 x evening slots
Go to huggingface.co/learn/nlp-course . Complete Chapter 1. It shows you the high-level API: <code>pipeline('ner')</code> runs a full NER model in 3 lines. This is the abstraction you will eventually replace with your own fine-tuned model.		
week 2 (transformers + fine-tuning setup, ~14 hrs) <i>understanding what you are working with</i>		
HF Course Chapter 2 — using the library	~5 hrs	weekday evenings
Covers <code>AutoTokenizer</code> , <code>AutoModel</code> , inputs/outputs, logits. You will learn how text becomes numbers (tokenisation) and how numbers become predictions. Critical: understand what <code>input_ids</code> , <code>attention_mask</code> , and <code>token_type_ids</code> are. These appear in every training script.		
HF Course Chapter 3 — fine-tuning with Trainer API	~6 hrs	weekend + 2 evenings

This is the most important chapter for your project. You will fine-tune BERT on a sentiment dataset. The code structure — load dataset, tokenise, create Trainer, call `trainer.train()` — is identical to what you will use for NER. Run the code, do not just read it.

HF Course Chapter 4 — sharing models + Datasets

~3 hrs

weekday
evenings

Learn the Datasets library. You will use it to load and preprocess your bank SMS data. Also covers pushing to HuggingFace Hub — useful for saving checkpoints remotely.

week 3 (tokenisers + data labelling, ~15 hrs)

the part most tutorials skip — and the hardest part of your project

HF Course Chapter 6 — tokeniser algorithms

~5 hrs

weekday
evenings

Covers BPE, WordPiece (used by BERT/TinyBERT), and Unigram. The critical concept: WordPiece splits words into subword tokens ('debited' → 'de', '##bit', '##ed'). Your NER labels must align to subwords, not words. Chapter 6 explains exactly how to handle this alignment — it is the most common source of NER bugs.

Start labelling your SMS data with BIO tags

~6 hrs

weekend
(block time)

Label your 50–100 SMS messages using this format: each token gets one label from {B-AMOUNT, I-AMOUNT, B-ACCOUNT, I-ACCOUNT, B-TYPE, B-DATE, O}. Use Label Studio (free, local) or a spreadsheet. You need ~300–500 labelled samples minimum before training. 1000 is better. This is your rate-limiting step.

Set up Google Colab for training

~2 hrs

weekend

Create a Colab notebook. Colab's free tier gives you a T4 GPU — enough for TinyBERT fine-tuning. Install transformers, datasets, accelerate. Test that a basic Trainer run completes without error. Do not start actual training yet.

Read HF token classification guide

~2 hrs

evening slots

Go to huggingface.co/docs/transformers/tasks/token_classification. This is the practical NER fine-tuning reference — different from the course material. Read it once end-to-end. You will return to it constantly during Week 4.

week 4 (NER fine-tuning, ~14 hrs)

HF Chapter 7 + fine-tune on your own data

HF Course Chapter 7 — token classification section

~6 hrs

weekend
morning +
evenings

Complete the NER section of Chapter 7. You will fine-tune DistilBERT on the CoNLL-2003 dataset (standard NER benchmark). This is practice — not your final model. Run every code cell. Understand the `DataCollatorForTokenClassification` and why it handles padding differently from standard classification.

Fine-tune TinyBERT on your labelled bank SMS data

~6 hrs

weekend +
evenings

Replace the CoNLL dataset with your labelled SMS data. Replace DistilBERT with huggingface.co/huawei-noah/TinyBERT_General_4L_312D. Run training on Colab. Evaluate on a held-out 20% of your data. Expected F1 on first run: 0.5–0.7. That is fine — you will improve it.

Iterate on label quality

~2 hrs

any slot

Look at where the model makes mistakes. Most errors come from inconsistent labelling, not model weakness. Re-label ambiguous examples. Re-train. Even 2–3 labelling passes significantly improve F1.

week 5 (data expansion + model improvement, ~12 hrs)

more data beats a better model — always

Expand your labelled dataset to 500–1000 samples

~6 hrs

distributed
across week

This is the highest-leverage thing you can do. Collect SMS messages from friends, family, or public datasets. Kaggle has Indian bank SMS datasets. Label everything consistently. More labelled data for NER nearly always beats hyperparameter tuning.

Experiment: TinyBERT vs MiniLM

~4 hrs

weekend

Fine-tune huggingface.co/sentence-transformers/all-MiniLM-L6-v2 on the same data. Compare F1 scores. MiniLM is ~22 MB quantised vs TinyBERT's ~15 MB — pick based on accuracy vs size tradeoff for your specific data.

Add LoRA for faster iteration (optional)

~2 hrs

optional

If Colab training feels slow or you are running out of free GPU hours, install peft and wrap your model with LoraConfig. LoRA fine-tunes only a small fraction of parameters — training becomes 3–5x faster with minimal accuracy loss.

week 6 (onnx export + quantisation, ~8 hrs)

from PyTorch model to mobile-ready binary

Install HuggingFace Optimum

~1 hr

any evening

`pip install optimum[onnxruntime]`. This is the HF library for model export and optimisation. Read the Optimum ONNX export documentation page once: huggingface.co/docs/optimum/exporters/onnx/usage_guides/export_a_model

Export your fine-tuned model to ONNX

~2 hrs

evening

Run: `optimum-cli export onnx --model ./your_finetuned_model --task token-classification ./onnx_output/`. This produces `model.onnx` plus tokenizer files. Verify the export ran without errors by loading it in Python: `from optimum.onnxruntime import ORTModelForTokenClassification`.

Quantise to INT8

~2 hrs

evening

Three lines of Python: `from onnxruntime.quantization import quantize_dynamic, QuantType; quantize_dynamic('model.onnx', 'model_quant.onnx', weight_type=QuantType.QInt8)`. Check the file size — TinyBERT should be ~15 MB at this point. Run inference on 5 test SMS messages and confirm outputs match your PyTorch model.

Bundle assets for React Native

~3 hrs

weekend

Your ONNX bundle needs three files: `model_quant.onnx`, `tokenizer.json`, `tokenizer_config.json`. Place them in your Expo project's `assets/ml/` folder. Confirm they are included in your EAS build by checking `app.json` asset patterns.

week 7–8 (react native integration, ~14 hrs)

ship it

Install and configure onnxruntime-react-native

~3 hrs

weekend

`npm install onnxruntime-react-native`. For Expo, you need a custom dev client (`expo install expo-dev-client`). ONNX Runtime is a native module — it does not work in Expo Go. You already have EAS configured; run `eas build` for a dev client build and test on your physical device.

Write the inference function

~4 hrs

evenings

Create a `useMLParser` hook. It loads the model on mount (`InferenceSession.create`), runs tokenisation (load `tokenizer.json` manually or use a JS tokeniser library), calls `session.run()` with your input tensors, and returns the raw logits. This is ~30–50 lines of TypeScript.

Post-process NER output to JSON	~3 hrs	evenings
Convert the raw per-token label predictions to structured data. Collect consecutive B-/I- tokens of the same type into single field values. Map label IDs back to names. Return: { amount: '2000', account: 'XX3423', type: 'debit', date: '12 Apr' }. This is plain TypeScript logic.		
Wire up the hybrid parser in kharcha	~4 hrs	weekend
Final integration: try transaction-sms-parser first. If it returns null/undefined for any field, run the ML model for those fields. Log which path each SMS took. Ship. Your on-device parser is now a two-layer system with deterministic speed for known banks and ML coverage for unknowns.		

4. fitting this into your actual days

Two study slots on weekdays. The 5–6 PM slot is short, good for passive learning (watching videos, reading docs). The 10 PM–midnight slot is for active work (writing code, running notebooks, labelling data). Weekends are your heavy lifting days.

Slot	Time	Best for	Avoid
Weekday pre-training (if training 7–8 PM)	5–6 PM (~1 hr)	Watch fast.ai/HF course videos. Read chapter pre-training.	Active coding. Reading yesterday's notes.
Weekday post-dinner	10 PM–12 AM (~2 hrs)	Run Colab notebooks. Label data. Write inference code. Debug training errors.	Staring at a single problem for too long. Late to context-switch.
Weekend mornings	9 AM–12 PM (~3 hrs)	Longest uninterrupted block. Use for active fine-tuning.	Passive video/data/labelling-spikes.
Wednesday WFH	Lunch break + evening	Extra flexibility. Good day to run a full Colab training job.	Nothing specific.

Training nights (boxing 7–8 PM, gym 7:30 PM): your 5–6 PM study slot becomes the prep window. Do not try to squeeze in coding before boxing — passive learning only. The 10 PM slot still works fine after training; you are mentally wound down, which actually suits labelling tasks and code review over new concept learning.

5. bio tagging reference

This is the labelling schema for your bank SMS NER model. Every person who labels data (even just you) must follow this consistently. Inconsistent labels are the primary cause of low F1 — not model weakness.

Tag	Meaning	Examples
B-AMOUNT	Beginning of a monetary amount	INR, Rs, 2000.00, 2,500
I-AMOUNT	Continuation of an amount	500 (when split: 'Rs' 'B-AMOUNT' '500' 'I-AMOUNT')
B-ACCOUNT	Beginning of account/card number	XX3423, XXXX1234, A/C
I-ACCOUNT	Continuation of account reference	ending (in 'A/C ending 3423')
B-TYPE	Transaction type	debited, credited, spent, received, withdrawn
B-BALANCE	Beginning of remaining balance	Avl Bal, balance, Bal
I-BALANCE	Continuation of balance	Rs (in 'Bal Rs 5000')
B-DATE	Date or time reference	12-Apr, 12/04/26, today
B-MERCHANT	Merchant/payee name	Swiggy, Amazon, IRCTC
I-MERCHANT	Continuation of merchant name	India (in 'Amazon India')
O	Outside — not a named entity	Dear, Customer, your, is, has, been

example labelled SMS

Input SMS:

Dear Customer, INR 2,000.00 debited from A/C XX3423 on 12-Apr. Avl Bal INR 15,432.10. —HDFC Bank

Token-level labels:

Dear	Customer	,	INR	2,000.00	debited	from	A/C
O	O	O	B-AMOUNT	I-AMOUNT	B-TYPE	O	B-ACCOUNT
XX3423	on	12-Apr	.	Avl	Bal	INR	15,432.10
I-ACCOUNT	O	B-DATE	O	B-BALANCE	I-BALANCE	I-BALANCE	I-BALANCE

6. resources

core learning (in order)

Resource	URL	Cost	Why
fast.ai Practical Deep Learning Lesson 4	course.fast.ai	Free	Start here — top-down, sees the whole picture first
HuggingFace NLP Course Ch 1–4	huggingface.co/learn/nlp-course	Free	Core fine-tuning pipeline
HF NLP Course Ch 6 — Tokenisation	huggingface.co/learn/nlp-course/chapter6	Free	Critical for NER alignment
HF NLP Course Ch 7 — Token Classification	huggingface.co/learn/nlp-course/chapter7	Free	NER fine-tuning hands-on

reference docs (bookmark, use constantly)

Resource	URL	Cost	Why
HF Token Classification Task Guide	huggingface.co/docs/transformers/tasks/token_classification	Free	Practical NER reference
HF Optimum ONNX Export Guide	huggingface.co/docs/optimum/exporters/onnx/usage_guides/export_a_model	Free	Export pipeline
onnxruntime-react-native README	github.com/microsoft/onnxruntime/tree/main/js/react_native	Free	RN integration
transaction-sms-parser npm	npmjs.com/package/transaction-sms-parser	Free	Your regex-first layer

models (on huggingface hub)

Resource	URL	Cost	Why
huawei-noah/TinyBERT_General_4L_312D	huggingface.co	Free	Primary model choice — 15 MB quantised
sentence-transformers/all-MiniLM-L6-v2	huggingface.co	Free	Fallback — 22 MB quantised, slightly more accurate
distilbert-base-uncased (for practice only)	huggingface.co	Free	Use in HF Course exercises only, too large for production

tooling

Resource	URL	Cost	Why
Label Studio	labelstud.io	Free OSS	Local NER data labelling — install with pip install label-studio
Google Colab	colab.research.google.com	Free	T4 GPU — sufficient for TinyBERT fine-tuning
HuggingFace Optimum	github.com/huggingface/optimum	Free	ONNX export: pip install optimum[onnxruntime]

optional depth (post-ship)

Resource	URL	Cost	Why
----------	-----	------	-----

Karpathy — Neural Networks: Zero to Hero	karpathy.ai/zero-to-hero	Free	Best ML fundamentals resource — do this after shipping
Stanford CS224N Lectures 8–11	youtube.com	Free	Transformers / attention / fine-tuning depth — post-ship
Made With ML	madewithml.com	Free	MLOps for software engineers — relevant for infra pivot

7. on google mlcc

Short answer: skip it.

Claim	Reality
MLCC gives you ML foundations before HF Course	fast.ai Lesson 4 gives you better foundations in 8 hrs. MLCC's 15–25 hrs teach regression
You need to understand gradient descent before fine-tuning	You do not. The Trainer API abstracts it. You will naturally build intuition by watching loss
MLCC is updated and modern (Nov 2024 refresh)	The refresh added an LLM conceptual overview. It still has zero hands-on fine-tuning and
MLCC is a detour you cannot afford	At 14–16 hrs/week study capacity, MLCC costs you 1.5–2 weeks with zero output applica

8. what your onnx experience actually gives you

The research document framed your HackerEarth ONNX work as giving you a head start on ONNX export. That is partially wrong, and it matters for how you plan your time.

What you did at HackerEarth	What you need for kharcha
Loaded a pre-built Silero VAD .onnx model using onnxruntime-web in a browser	Exported a PyTorch model to ONNX format using HuggingFace Optimum
Called InferenceSession.create() and session.run() with audio tensor inputs	Quantise the ONNX model to INT8 using onnxruntime.quantization
Handled Float32Array inputs and interpreted binary (speech/silence) outputs	Loaded model in React Native using onnxruntime-react-native (different package from
Zero exposure to model training, export pipeline, or quantisation	Post-process NER logits (multi-class per token) into BIO label sequences

The net effect: your HackerEarth experience means onnxruntime's API is not alien to you, and you have debugged a real inference pipeline before. But you are still learning ONNX export and quantisation from scratch in Week 6. Budget for it accordingly — do not assume familiarity that does not exist.

9. decision checklist

✓ / ✗	Decision
DO	Use onnxruntime-react-native, not react-native-executorch
DO	Use TinyBERT (primary) or MiniLM-L6 (fallback), not DistilBERT or T5
DO	Install transaction-sms-parser first as your regex layer — before any ML work
DO	Start collecting SMS data now — labelling is your rate-limiting constraint
DO	Use Google Colab for training (free T4 GPU, sufficient for TinyBERT)
DO	Use Label Studio locally for BIO annotation
DO	Skip MLCC entirely
DO	Do fast.ai Lesson 4 before starting HF Course
DO	Chapters to complete: 1, 2, 3, 4, 6, 7 (skip 5, 8–12 initially)
DO	Export via HuggingFace Optimum CLI, not raw torch.onnx.export
DO	Quantise to INT8 — check model size is under 20 MB after quantisation

DO	Expo custom dev client is required for onnxruntime-react-native (not Expo Go)
DON'T	Do NOT use ExecuTorch / .pte format for this project
DON'T	Do NOT use DistilBERT in production — it exceeds your 50 MB budget
DON'T	Do NOT skip the tokeniser chapter (Chapter 6) — NER alignment bugs start here
DON'T	Do NOT start with a pure ML parser — regex-first hybrid is more accurate and debuggable

This plan will need revision once you hit actual blockers — data labelling quality, Colab quota limits, onnxruntime-react-native version conflicts. Treat it as a working document. The architecture decisions in Section 1 are the ones that cannot easily be undone later; everything else is adjustable.